Sesión 04 — Casos

Validación de seguridad en integración continua: Retail Andina, Financiera Los Andes y Seguros Sur

Jorge Chamorro · Resolución de los tres casos del archivo «Sesión 04 - Casos.pdf», aplicando la teoría del Capítulo 3 (sesiones 3 y 4), el material de consulta del Examen 1 y las notas de la clase.

Marco teórico aplicado

Las tres resoluciones usan las mismas cinco ideas de la sesión; se enuncian una vez aquí para no repetirlas en cada caso.

1. Cuándo un control de seguridad pertenece a la integración continua

Un control entra a esta etapa si cumple los cuatro criterios: **rápido** (minutos), **determinista** (misma entrada, mismo resultado), **acotable al cambio** (puede evaluar solo el código nuevo, no todo el repositorio) y **no exige ambiente desplegado**. Cumplen los cuatro: detección de secretos, análisis de composición (SCA), análisis estático de seguridad (SAST) y análisis de configuración/IaC. El análisis dinámico (DAST, ethical hacking) no cumple el cuarto y pertenece a una etapa posterior.

2. Política del gate

- **Detiene lo nuevo, inventaría lo heredado.** Bloquear todo desde el primer día deja el pipeline en rojo permanente y el equipo desactiva el control; registrar siempre y no detener nunca produce un reporte que nadie abre.
- **Umbral por tipo de control**, no uno solo: un secreto detiene siempre; una dependencia de severidad media con parche puede registrarse y planificarse.
- **La condición se escribe antes del hallazgo** y queda versionada en el repositorio.
- **La excepción** lleva cuatro datos: identificador del hallazgo, razón, responsable que la autoriza y fecha de vencimiento; al vencer el pipeline vuelve a detenerse. Una credencial **nunca** se exceptúa: se rota y se limpia del historial. El número de excepciones vigentes es la métrica de calibración del umbral.

3. Severidad no es prioridad

La severidad ordena; priorizar exige **alcanzabilidad** (¿se ejecuta la función vulnerable?), **exposición** (¿está en un servicio expuesto a internet?) y **existencia de corrección** (¿hay versión corregida?). Calidad y seguridad son dos compuertas distintas: la primera pregunta «¿está bien construido?», la segunda «¿es explotable?», y la excepción de seguridad la autoriza idealmente el equipo junto con quien responde por el riesgo.

4. Qué métrica DORA mueve la integración continua

Mueve sobre todo **lead time para cambios**, **frecuencia de despliegue** y **tasa de fallas en cambios**. El **tiempo de restauración** rara vez depende de la integración continua (lo señaló el profesor en el cierre del Laboratorio 2), así que no se declara como métrica movida salvo que el caso lo justifique. Un pipeline puede volverse algo más lento y más seguro: es un trade-off aceptable.

5. Marco de referencia (NIST SP 800-218)

SCA → **PW.4** (reutilizar software existente y verificar componentes de terceros); SAST y secretos → **PW.7** (revisar el código legible); análisis dinámico → **PW.8** (probar el código ejecutable); identificación continua de vulnerabilidades sobre lo ya publicado → **RV.1**.

Caso 1 · Retail Andina

Diagnóstico. El recurso escaso del flujo es la ventana del único ambiente de pruebas que comparten cinco equipos (el ERP solo existe en dos ambientes). El escaneo de seguridad lo corre infraestructura **contra la aplicación ya desplegada**, ocupa una ventana completa, la pierde otro equipo y el resultado llega por correo días después. Además revisa dependencias, código y configuración **sin distinguir qué necesita la aplicación corriendo**. Los congelamientos de campaña acumulan lotes grandes, donde los defectos se mezclan.

1. ¿Qué parte del escaneo necesita la ventana y cuál no?

Parte del escaneo	¿Necesita ambiente desplegado?	Dónde debe vivir
Dependencias (SCA)	No: se evalúa el árbol de dependencias contra bases de vulnerabilidades, con el archivo de bloqueo.	Pipeline, en el pull request y antes de publicar.
Código (SAST)	No: recorre el código fuente sin ejecutarlo.	Pipeline, acotado al código nuevo del cambio.
Secretos en repositorio e historial	No.	Pipeline, en cada cambio; el historial completo con checkout total.
Configuración declarada (plantillas de infraestructura, Dockerfile, archivos de configuración del servidor como código)	No: se evalúa contra reglas escritas como código.	Pipeline (análisis de configuración e IaC).
Configuración y comportamiento del servidor **en ejecución**, pruebas dinámicas contra la aplicación respondiendo	Sí: necesita la aplicación desplegada.	Ventana del ambiente, sobre un artefacto que ya pasó los gates del pipeline.

Con esto la ventana solo se usa para lo que realmente exige el ambiente. Dependencias, código y secretos dejan de consumirla, y el feedback llega al desarrollador al hacer el commit (con el contexto fresco) y no días después por correo. Eso reduce la espera del VSM y evita que otro equipo pierda su ventana.

2. Controles que pasan al pipeline: ¿cuáles detienen y cuáles solo registran?

Control	Detiene	Solo registra
Secretos	Siempre: cualquier credencial nueva en el cambio. Para una ya presente en el historial, se rota.	Nada: un secreto nunca se registra sin acción.
SCA	Dependencia **nueva** con vulnerabilidad crítica o alta que tenga versión corregida.	Severidad media o baja; hallazgos sin corrección disponible; todo lo heredado (se inventaría y se planifica).
SAST	Patrón explotable de severidad alta en el código **nuevo** del cambio.	Hallazgos preexistentes y de severidad baja.

Control	Detiene	Solo registra
Configuración / IaC	Configuraciones críticas declaradas (por ejemplo, exposición pública o datos sin cifrar).	Recomendaciones de endurecimiento menores.

Se aplican umbrales distintos por control, no uno solo, y todo se mide sobre el código nuevo («detiene lo nuevo, inventaría lo heredado»): así los cinco equipos no heredan un pipeline en rojo desde el primer escaneo.

3. Pase de emergencia en plena campaña y el gate detiene por una vulnerabilidad

No se desactiva ni se salta el gate. En campaña el costo de un incidente es máximo y el gate es la validación previa más barata; saltarlo una vez enseña al equipo a saltarlo siempre. Se actúa por pasos, con una vía de emergencia que ya estaba definida antes del hallazgo:

- **Triage en minutos:** ¿el hallazgo lo introduce este cambio o es heredado? Si es heredado, el gate no debía detenerlo (se revisa la línea base). Si es nuevo, se evalúa severidad, alcanzabilidad, exposición y si existe corrección.
- **Si hay corrección rápida** (subir una versión, quitar un patrón), se aplica y el pase continúa: es lo más barato y lo más seguro.
- **Si no hay corrección o no es alcanzable,** se registra una **excepción de emergencia** con sus cuatro datos: identificador del hallazgo, razón, responsable que la autoriza (Seguridad, junto al líder del equipo) y vencimiento corto (propuesta: no más allá del cierre de la campaña). Se acompaña de una mitigación (feature flag, monitoreo reforzado de ese componente).
- **Si es un secreto**, no hay excepción: se rota la credencial y el pase continúa con la nueva.
- **Después de la campaña:** postmortem del hallazgo y revisión del umbral si fue un falso positivo.

Por qué: la excepción con responsable y vencimiento conserva la trazabilidad, no convierte la urgencia en un bypass permanente y, al vencer, el pipeline vuelve a detener y se decide de nuevo con la información de ese momento.

4. ¿Qué métrica DORA movemos?

Principalmente **lead time para cambios**: el escaneo deja de esperar una ventana compartida y deja de entregar su resultado días después. Como efecto secundario, sube la **frecuencia de despliegue** (la ventana deja de ser el límite del flujo y se acumulan menos lotes en los congelamientos) y baja la **tasa de fallas en cambios** (menos vulnerabilidades llegando al ambiente). El tiempo de restauración no se ve afectado directamente.

Caso 2 · Financiera Los Andes

Diagnóstico. El gobierno está al final del flujo: un cambio termina de construirse y espera la cola semanal del comité. La lista de seguridad vive en una hoja de cálculo, se llena a mano con el código ya congelado, **un solo analista** la conoce y la evidencia de auditoría se arma a mano días antes. Hoy una parte automatizable y otra que exige criterio se revisan igual, con el mismo esfuerzo y en el mismo momento.

1. ¿Qué controles se automatizan y cuáles se quedan con el analista?

Control	Decisión	Por qué
Versiones de dependencias contra vulnerabilidades conocidas	Pipeline (SCA)	Determinista y repetible. Requiere archivo de bloqueo en cada stack (Java, .NET, React Native).
Patrones de código ya catalogados	Pipeline (SAST)	El caso dice que ya están catalogados: una regla escrita corre igual todas las veces.
Credenciales en el repositorio	Pipeline (secretos, sobre el cambio y sobre el historial)	Es el control más barato y de hallazgos menos discutibles.
Revisión de diseño y de cambios de alto riesgo (por ejemplo, flujos que tocan datos de clientes o desembolsos)	Analista	Exige criterio del negocio y de riesgo; ninguna herramienta lo cubre (PW.1–PW.2).
Evaluación de excepciones y priorización contextual (alcanzabilidad, exposición)	Analista	Es una decisión de riesgo con responsable, no una verificación.

El analista pasa de **leer la misma lista en cada entrega** a ser quien **define y gobierna la política** (umbrales, excepciones, qué cambios son de alto riesgo) y revisa solo lo que requiere criterio. Además se elimina el punto único de falla: la parte automatizada corre aunque el analista esté de vacaciones, y la lista deja de depender de su memoria porque queda escrita como política versionada. Los ejemplos de controles manuales son típicos; el caso no enumera la lista completa.

2. Controles automatizados: ¿cuáles detienen y cuáles solo registran?

Control	Detiene	Solo registra
Secretos	Siempre. Credencial nueva: se rota y se limpia del historial; nunca se exceptúa.	—
SCA	Vulnerabilidad nueva de severidad crítica o alta con versión corregida disponible.	Media o baja, sin corrección, y todo lo heredado en el inventario.
SAST	Patrones catalogados de severidad alta en el código nuevo.	Hallazgos heredados y de severidad baja.

Toda excepción lleva identificador, razón, responsable y vencimiento; en una financiera, el responsable debe incluir al área de riesgos. Cada ejecución queda registrada con versión, resultado y fecha: eso reemplaza las carpetas armadas a mano y convierte la auditoría en una consulta (evidencia continua).

El comité se conserva —en contextos regulados la aprobación se mantiene— pero revisa la evidencia que el pipeline ya produjo y se concentra en los cambios de alto riesgo.

3. El pipeline aprobó el release; tres semanas después la auditoría interna halla una vulnerabilidad que la lista del analista sí habría detectado

¿Qué pasó? Hay varias causas posibles, que no se excluyen, y todas son fallas del proceso:

- **La base de vulnerabilidades cambia con el tiempo.** El pipeline evaluó contra la base del día del release; en tres semanas aparecieron vulnerabilidades nuevas. Un release que estaba limpio deja de estarlo sin que el código cambie (por eso se dice que son «vivas»). Fue la explicación discutida en clase.
- **La vulnerabilidad cae en la parte no automatizable.** Si el analista dejó de revisar el release porque «lo aprobó el pipeline», el pipeline nunca cubrió ese control.
- **La regla o el umbral no cubría ese caso** (falso negativo) o el hallazgo había sido registrado y nadie lo planificó.

¿Culpa del analista o de la herramienta? De ninguno: del **proceso**. La herramienta hace lo que se le configuró con la información que tiene ese día, y el analista no estaba en el flujo. Buscar un culpable cierra el postmortem con un nombre y no corrige nada.

Qué cambian para la próxima:

- Re-evaluar de forma **programada** el inventario de componentes de lo ya desplegado contra la base actualizada, para responder «¿estoy afectado?» sin esperar a la siguiente auditoría (práctica RV.1 del NIST).
- Hacer explícito qué cubre la herramienta y qué cubre el analista, y que el analista revise los cambios de alto riesgo aunque el pipeline esté en verde.
- Analizar la causa raíz (RV.3): si fue un falso negativo, ajustar la regla o el umbral y versionar el cambio.
- Escribir la lista completa como política documentada y formar a un segundo analista.

4. ¿Qué métrica DORA movemos?

Frecuencia de despliegue y **lead time para cambios**: la seguridad deja de esperar al final en una cola semanal y con el código congelado, y el release mensual se puede partir en lotes más chicos. El comité sigue existiendo por ser un contexto regulado, de modo que el tope práctico lo marca su cadencia. En segundo lugar mejora la **tasa de fallas en cambios**, al detectar los hallazgos en el commit y no en la auditoría.

Caso 3 · Seguros Sur

Diagnóstico. El portal y el app tienen pipeline en GitHub Actions (instala, construye, corre pruebas unitarias y publica) que corre en **cada push a cualquier rama** y toma seis minutos, **sin ninguna validación de seguridad**. El core de pólizas se construye y despliega a mano. La seguridad llega por una auditoría externa trimestral sobre código que ya cambió; entre informes, nadie revisa una dependencia antes de que entre. El framework está dos versiones mayores atrás y bloquea la actualización de otras librerías. El retrabajo es alto y el tablero no distingue funcionalidad de corrección.

1. ¿Qué controles activan y cuáles descartan?

Control	Decisión	Por qué
Detección de secretos	Activa	Hoy no existe y nadie revisa el historial. Es barato, rápido y de hallazgos indiscutibles.
Análisis de composición (SCA)	Activa	Es el dolor concreto del caso: el auditor reporta vulnerabilidades en dependencias semanas tarde y nadie revisa las dependencias nuevas. Requiere archivo de bloqueo (se verifica que exista).
SAST	Activa, en modo código nuevo	Cumple los cuatro criterios. Se limita al código nuevo para no cargar con el código heredado; arranca registrando para calibrar.
Configuración / IaC	Descarta por ahora	El caso no describe plantillas de infraestructura declaradas; el core se despliega a mano. No hay objeto que analizar.
Análisis dinámico	Descarta	Exige la aplicación desplegada y respondiendo; no pertenece a la integración continua.

El **core de pólizas** queda fuera de esta etapa: sin pipeline no hay dónde ejecutar un control. Su paso a pipeline es un trabajo previo, no una decisión de umbral.

2. Para cada control activado: ¿en qué punto corre y qué pasa al encontrar algo?

Control	Punto de ejecución	Resultado	Condición
Secretos	Cada push (rápido) y como chequeo requerido del pull request; revisión del historial completo (checkout total) una vez al inicio y de forma programada.	Detiene	Siempre. Credencial nueva o histórica: se rota y se limpia; no admite excepción.
SCA	En el pull request (antes de fusionar), cada vez que cambia una dependencia, y como requisito antes de publicar. Además, una corrida programada contra la base actualizada.	Detiene lo nuevo; registra lo demás	Detiene una vulnerabilidad nueva crítica con corrección disponible. La corrida programada solo registra y abre ticket.
SAST	En el pull request, sobre el código nuevo.	Primeros sprints: registra; luego detiene	Después del período de calibración detiene hallazgos nuevos de severidad alta.

Los tres trabajos se ejecutan **en paralelo**, sin encadenarse entre sí, y la publicación depende de los tres (el gate); así la duración se mantiene en el orden de minutos y no se pierde el criterio de «rápido». Se cambia el disparador actual: validar en pull requests y publicar solo desde la rama principal, en vez de correr todo en cada push a cualquier rama. Aceptamos que el pipeline será algo más lento a cambio de ser más seguro.

3. Más de 400 hallazgos, 60 críticos, 35 de ellos solo corregibles migrando el framework: ¿qué umbral desde el lunes?

Umbral propuesto: el pipeline **detiene ante cualquier hallazgo crítico nuevo** (es decir, ausente de la línea base) que tenga versión corregida disponible. Los altos nuevos se registran durante los primeros sprints y pasan a detener cuando el equipo lo acuerde (el umbral se parte del valor actual y se sube por acuerdo). Medios y bajos solo se registran.

Cómo se arma:

- **Línea base congelada el mismo lunes:** la primera ejecución (400+ hallazgos, 60 críticos) se guarda como inventario heredado, cada hallazgo con su identificador. Lo heredado **no detiene**, pero **no se ignora**: se planifica.
- **Los 35 críticos que dependen del framework:** no se apaga la regla ni se los deja sin dueño; se registra una **excepción por hallazgo** (no por regla), con razón «solo se corrige migrando el framework, sin fecha», responsable (líder técnico del portal, con autorización de Riesgos/Seguridad) y vencimiento (propuesta: 90 días, alineado con el ciclo trimestral de la auditoría; al vencer se revisa de nuevo). Se acompaña de priorización por exposición y alcanzabilidad.
- **Los otros 25 críticos** se corrigen subiendo una versión: se planifican como trabajo visible en los próximos sprints, etiquetado como seguridad en el tablero (y no como historia nueva), para que el retrabajo deje de ser invisible.
- **Los 35 críticos son también el argumento de negocio de la migración** que hoy no tiene fecha: pasan de ser una opinión técnica a una deuda de seguridad medible.

Por qué este umbral: bloquear todo desde el lunes dejaría el pipeline rojo de forma permanente (hay hallazgos que no se pueden corregir) y el equipo desactivaría el control o aprendería a saltarlo. Registrar todo sin detener nada reproduciría la auditoría trimestral: un reporte que nadie abre y que reaparece en el siguiente informe. Detener lo nuevo corta el problema donde nace, sin depender de la migración. La métrica de control es el **número de excepciones vigentes** y su antigüedad: si crece mes a mes, el umbral está mal calibrado o falta capacidad de remediación.

4. ¿Qué métrica DORA movemos?

Tasa de fallas en cambios: una vulnerabilidad que llega a producción exige corrección o parche, y se detecta en el pull request y no semanas después. También reduce el **lead time** efectivo del cambio al evitar el retrabajo tardío sobre código que ya cambió (aunque el escaneo en el pull request añade unos minutos al flujo). Para que el retrabajo sea medible, conviene etiquetar en el tablero la corrección frente a la funcionalidad, como indicador propio (no es una de las métricas DORA vistas en el material). La frecuencia de despliegue no es la métrica principal de este caso.

Resumen comparativo

	Caso 1 · Retail Andina	Caso 2 · Financiera Los Andes	Caso 3 · Seguros Sur
Restricción dominante	Ambiente de pruebas único y ventana compartida	Gobierno al final del flujo y un solo analista	Sin validaciones y auditoría trimestral tardía
Qué se automatiza	SCA, SAST, secretos y configuración declarada; solo lo dinámico queda en la ventana	SCA, SAST y secretos; el criterio queda con el analista	Secretos, SCA y SAST; se descartan IaC y dinámico
Política del gate	Detiene lo nuevo; umbral por control	Detiene lo nuevo; excepción con riesgos como responsable	Línea base congelada; detiene críticos nuevos con corrección; excepción por hallazgo
Punto clave	Pase de emergencia: excepción, no bypass	El fallo es del proceso, no de una persona	Excepciones con vencimiento para lo que depende de la migración
DORA principal	Lead time	Frecuencia y lead time	Tasa de fallas en cambios

Fuentes y supuestos

Fuentes: diapositivas de la Sesión 04 (Integración continua II), notas de clase de la sesión, y el material de consulta del Examen 1 (capítulo 3, política del gate; anexo NIST SP 800-218). Los casos 1 y 2 se discutieron en clase; el Caso 3 no, por lo que su resolución es propia.

Valores que son propuestas y no datos del caso: los vencimientos de excepción (cierre de campaña; 90 días), el período de calibración de SAST, los ejemplos de controles manuales del Caso 2 y el reparto crítico/alto del umbral. Pueden cambiarse sin alterar la lógica de la política.